Reinforcement learning

Egor Slyachkus

Maxim Medvedev

October 8th, 2026

Three pillars of Machine Learning

Supervised Learning	**Unsupervised Learning**	**Reinforcement Learning**
Model learns from labeled training data to map inputs to specific, known outputs	Model analyses unlabeled data to discover hidden patterns or structures independently	Autonomous agents learn to make decisions in dynamic environments through trial-and-error rewards and penalties
· Regression, classification, object detection	· Clustering, customer segmentation, anomaly detection	· Robotics, algorithmic trading, videogame intelligence

Key differences of RL

· **Data Source.** Traditional ML relies on pre-collected, static historical datasets. RL generates its own data dynamically through live interaction

· **The Learning Signal.** Supervised gets direct labels and unsupervised gets no feedback at all. RL uses a distinct reward and penalty system based on environment's response

· **Nature of the Goal.** Traditional ML maps inputs to static outputs or clusters. RL optimizes a continuous, chronological sequence of decisions

· **Impact on the System.** Traditional ML models are passive observers and do not change the data they analyse. RL agents are active influencers, their actions directly change the next state of the environment

Why RL is more difficult than traditional ML

· **Delayed Reward Problem** ➔ need to consider the consequences of actions in the future

· **Agent affects the environment** ➔ actions change the next state

· **Sequential decision making** ➔ the goal is a chain of decisions, with each step coming from previous one

Solution: Model that considers a sequence of decisions and responses ➔ **Markov Decision Process**

Markov Decision Process (MDP)

Markov Property: Agent's decision depends **only on the current state**, not the full history of states and actions

Components:

- **State (S).** Current situation of the agent
- **Action (A).** Possible moves the agent can take at given state
- **Reward (R).** Immediate environment's feedback from actions
- **Transition Probability (P).** Likelihood of moving from one state to another

Goal: Find the **Policy (π)** that maximises total cumulative reward

Transition Probability and Markov Chains

Markov Property

$$P(s_{t+1} \mid s_t, s_{t-1}, \dots, s_0) = P(s_{t+1} \mid s_t)$$

Transition Matrix:

Current states (s), Next states (s'): Dry (D), Wet (W)
Action (a): Open umbrella (U), Do nothing (N)

$$\mathrm{P}(\mathrm{s}'|\mathrm{s},\mathrm{a}) = \begin{bmatrix} \text{From/To} & \text{Dry (D)} & \text{Wet (W)} \\ If\ a = U\text{:} & & \\ \quad D & 0.9 & 0.1 \\ \quad W & 0.7 & 0.3 \\ If\ a = N\text{:} & & \\ \quad D & 0.6 & 0.4 \\ \quad W & 0.2 & 0.8 \end{bmatrix}$$



Discounting of Total Reward

Total Reward **without** discounting

$$R(\tau) = r_{t+1} + r_{t+2} + r_{t+3} + r_{t+4} + \dots$$

Problem: Future rewards are uncertain ➔ need to prioritise immediate rewards

Total Reward **with** discounting

$$R(\tau) = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \gamma^3 r_{t+4} + \dots \quad \rightarrow \quad R(\tau) = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

$\gamma \approx 1$: Agent values long-term rewards (winning in chess)

$\gamma \approx 0$: Agent values short-term rewards (having profit in trading)

State vs. Observation

State (S). Agent has the complete information about the environment

Fully Observable MDP: agent knows S ➔ can plan optimally

- Board games (chess, checkers), grid navigation, inventory management

Observation (O). Agent only has observation, a partial description of the state

Partially Observable MDP: agent knows O ➔ must consider hidden states

- Card games (poker), trading, robotics, medical diagnosis

Action Space in RL

Discrete Action Space: Number of actions is a finite, limited set

- Platformer game (left, right, jump), chess (allowed moves)

Continuous Action Space: Number of possible actions is infinite

- Self-driving car (steering angle + acceleration), robot arm (joints movements)

Exploration vs. Exploitation

Exploitation: Using the best **known** actions to maximise the reward

- Going to the same favourite restaurant

Exploration: Trying new actions in hope of finding better strategies

- Visiting new restaurants

Trade-off: Agent must balance between exploring new options and exploiting known ones

Problem solving approaches

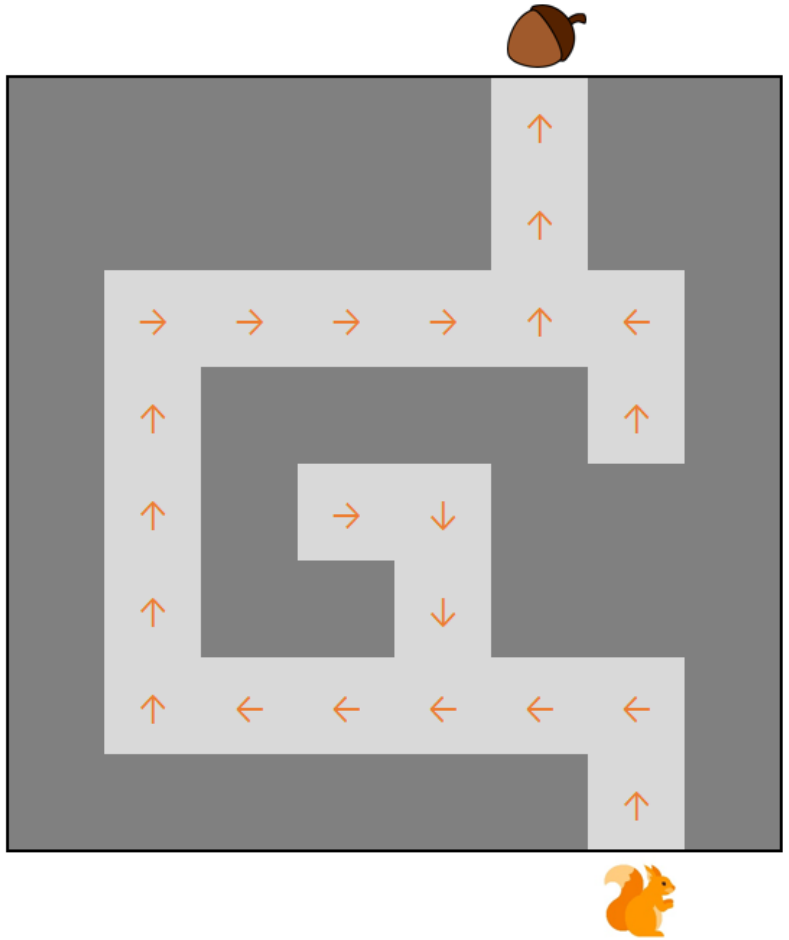
Policy-based method

Teaches the agent directly what actions to take in the current state

- **Deterministic:** Every state always gives the same single action $\pi(s) = a$

- **Stochastic:** States have probability distribution over actions

Action	Left	Right	Up	Down
$\pi(a|s_i)$	0.3	0.25	0.4	0.05



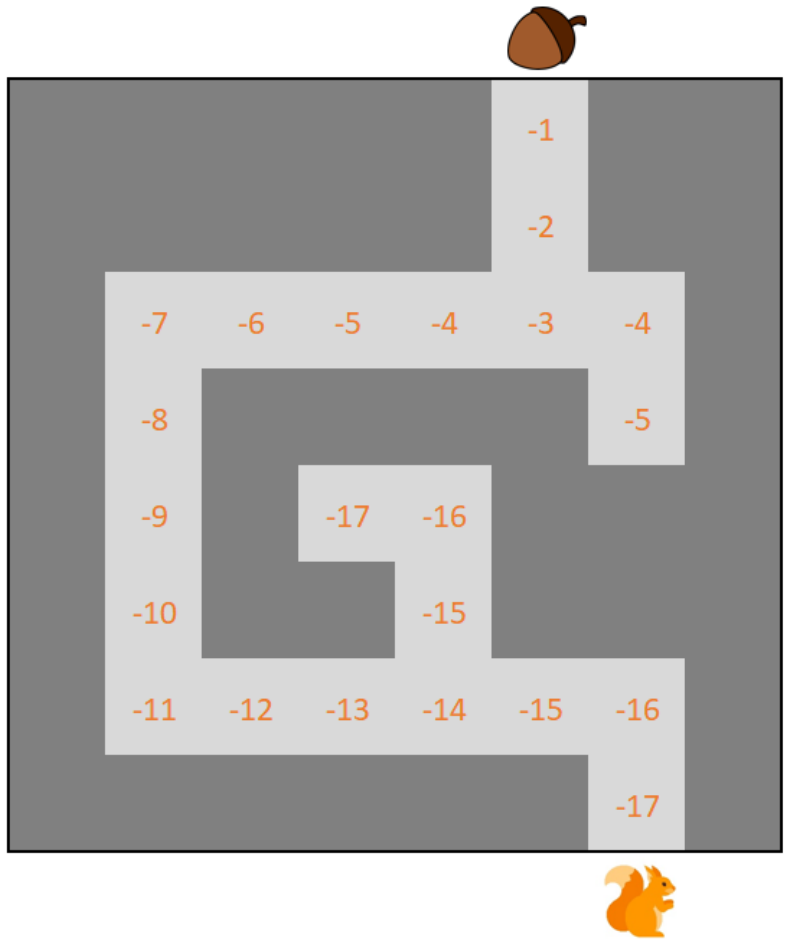
Problem solving approaches

Value-based method

Maps out the expected long-term value of being in each position

Higher state values mean more desirable positions under the policy

$V(s_i)$: Expected return from starting in state s_i and following the policy



Detailed Value-Based Method

Key idea: In value-based method, finding and **optimal value function** leads to having an **optimal policy**

State-value function

Expected return if agent starts in state **s** and follows policy forever

$$V_\pi(s) = E_\pi(G_t \mid S_t = s)$$

E_π – expected value, if policy is followed

G_t – cumulative reward

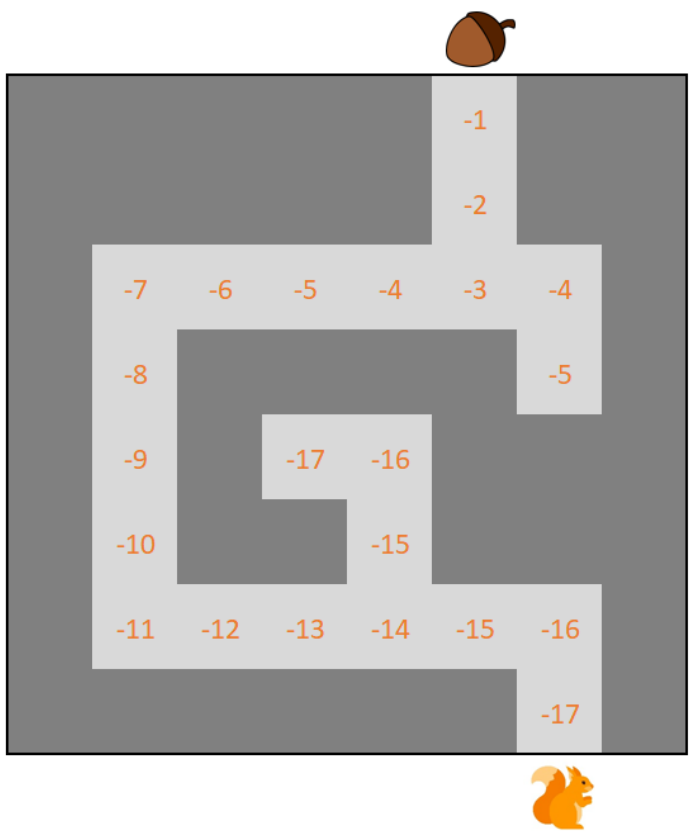
Action-value function

Expected return if agent starts in state **s**, takes **optimal action a** and follows policy forever

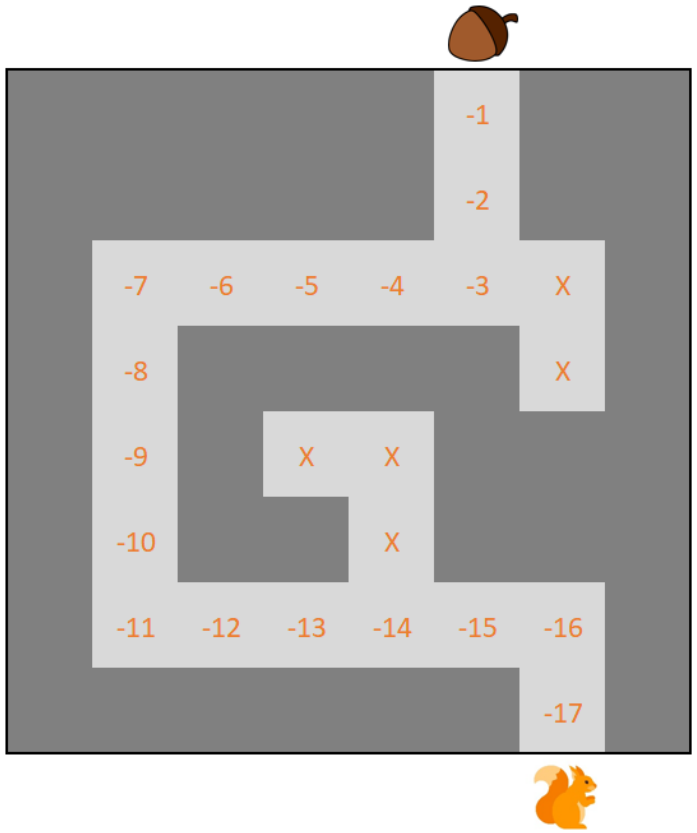
$$Q_\pi(s, a) = E_\pi(G_t \mid S_t = s, A_t = a)$$

Detailed Value-Based Method

State-value function V(s)



Action-value function Q(s, a)

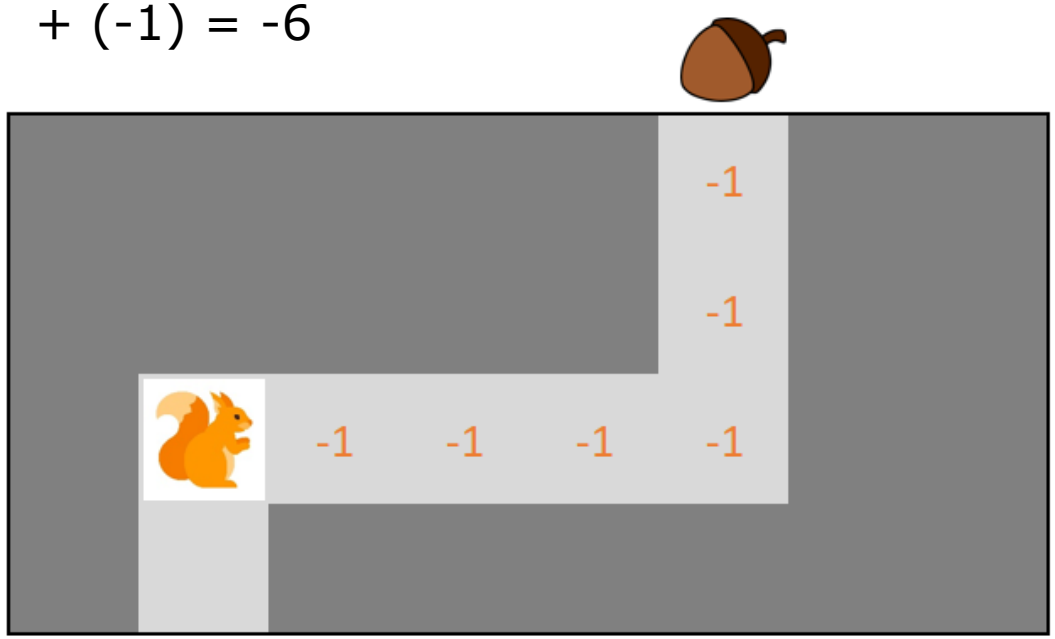


Calculating all values and summing rewards can be complex ➔ **Bellman equations** help with this

Value Calculation

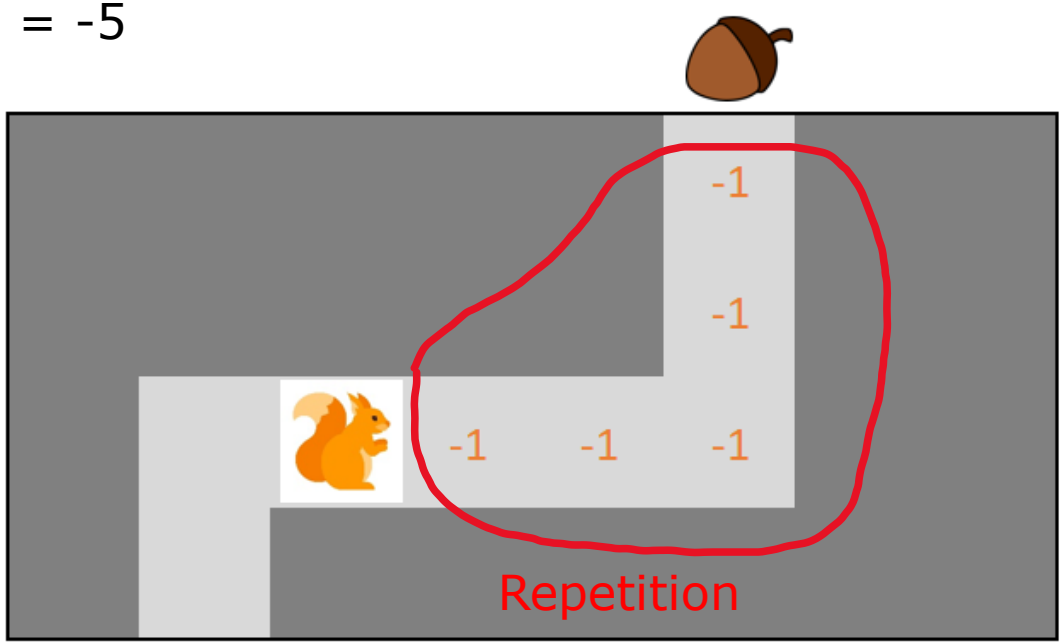
To calculate the value of a state, we need to compute the return, starting at that state and following the policy forever

$\mathbf{V(S_t)}$ = -1 + (-1) + (-1) + (-1) + (-1) + + (-1) = -6



→

$\mathbf{V(S_{t+1})}$ = -1 + (-1) + (-1) + (-1) + (-1) = = -5



Solution: To avoid repetition, calculate rewards recursively, starting from the end

Bellman Equations and Greedy Policy

Bellman equations solve the problem by:

State-Value: $V^*(s) = \max_a E_\pi(R_{t+1} + \gamma * V^*(S_{t+1} | S_t = s))$

Action-Value: $Q^*(s, a) = E_\pi(R_{t+1} + \gamma * \max_a, Q^*(S_{t+1}, a') \mid S_t = s, A_t = a)$

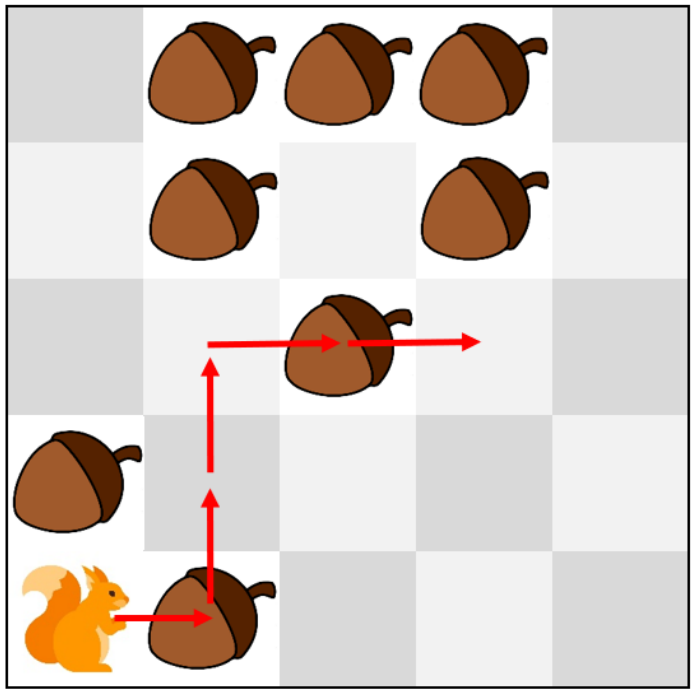
Greedy policy $\pi(s) = \arg\max_a Q^*(s, a)$

A policy that selects, at each state, the action with the highest expected cumulative discounted reward

Learning approaches: Monte Carlo

Agent waits until the end of the episode before learning, then calculates **actual** total return and uses it as a target for updating the state value

$$V(S_t) \leftarrow V(S_t) + \alpha\left(G_t - V(S_t)\right), \quad \alpha \text{ – Learning rate (e.g., 0.1)}$$



$$G_t = R_{t+1} + R_{t+2} + \cdots \quad \rightarrow \quad G_t = 1 + 0 + 0 + 1 + 0 = 2$$

Update $V(S_0)$:

Initial $V(S_0) = 0$ → New $V(S_0) = 0 + 0.1 * (2 - 0) = 0.2$

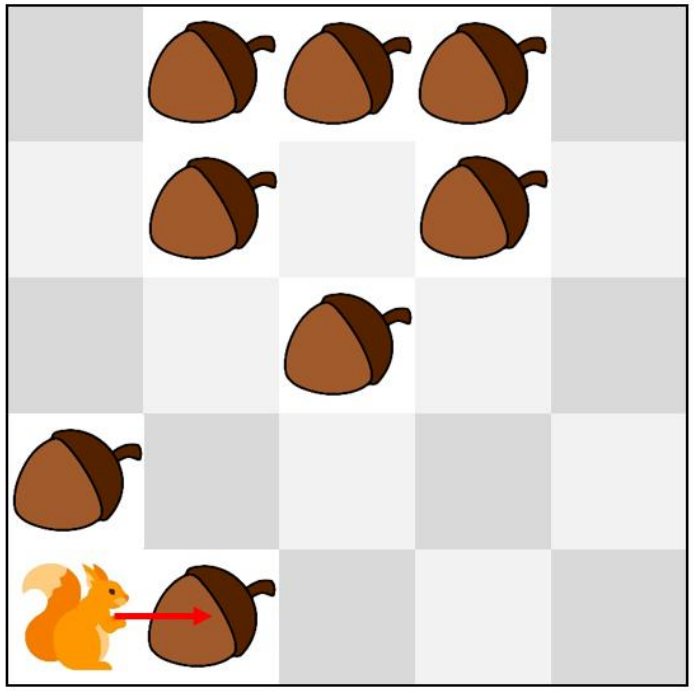
Pros: Accurate, uses true return

Cons: Slow, must wait until end of the episode to learn

Learning approaches: Temporal Difference

Agent updates value estimates at every step, without waiting for the end of the episode. It uses an **estimate** of the reward based on the immediate reward and next state's value

$$V(S_t) \leftarrow V(S_t) + \alpha\left(R_{t+1} + \gamma * V(S_{t+1}) - V(S_t)\right), \quad \alpha \text{ – Learning rate (e.g., 0.1)}$$



$R_{t+1} = 1$ $\gamma = 1$ (no discount)

Update $V(S_0)$: Initial $V(S_0) = 0$ ➔

➔ New $V(S_0) = 0 + 0.1 * (1 + 1 * 0 - 0) = 0.1$

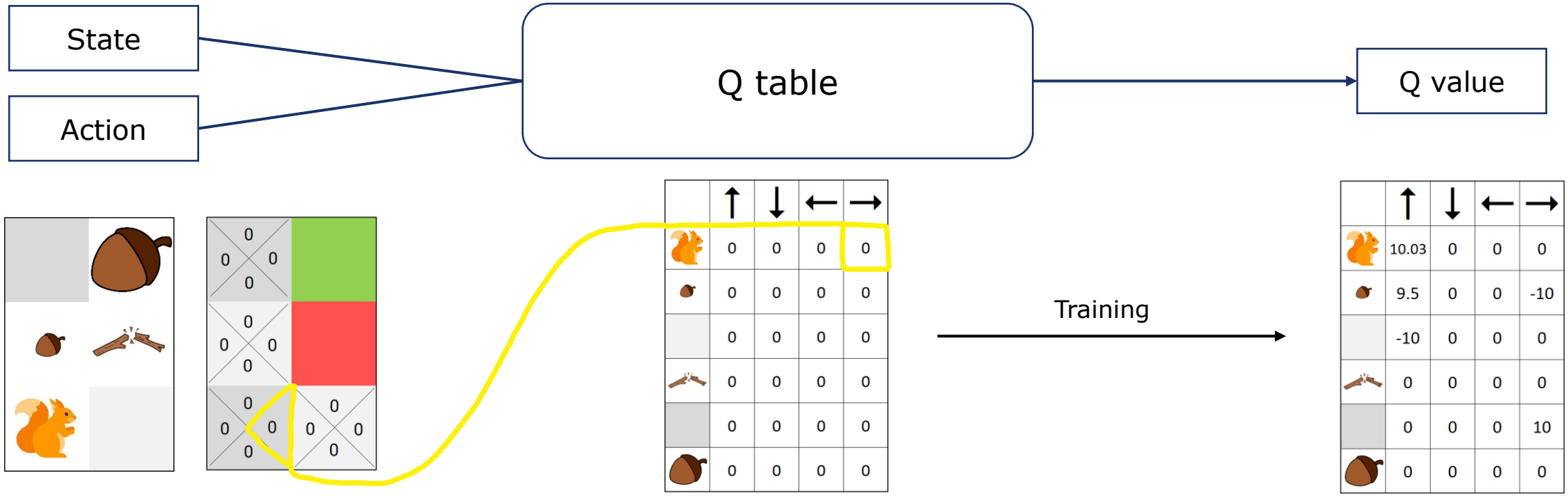
Pros: Faster learning, updates at every step

Cons: Uses an estimate of the return, not true return

Q-learning

Q-learning is a reinforcement algorithm, used to train the Q-function – the action-value function, that determines the expected cumulative reward of being at a state and taking a specific action

Q-function is encoded by a Q-table, where each cell corresponds to a state-action pair value

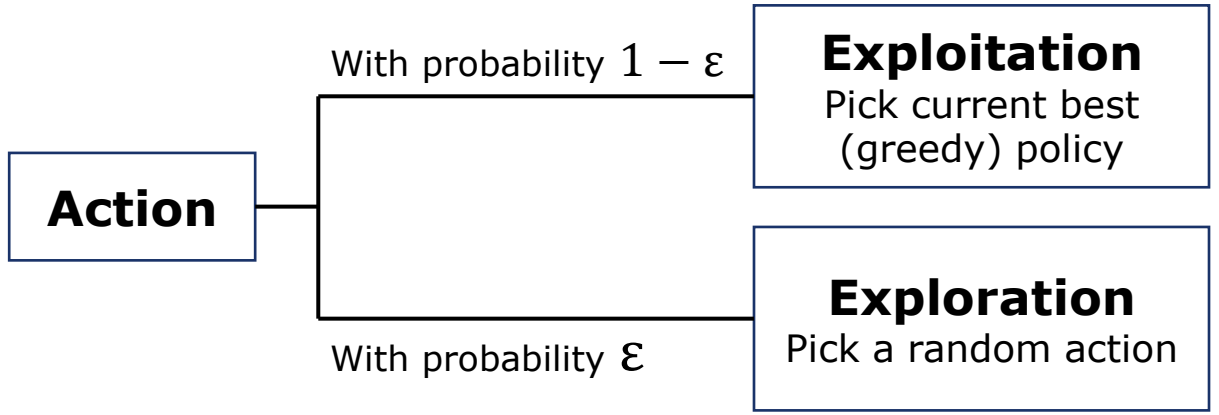


Q-learning algorithm

Step 1: Initialise Q-table

Start by setting all Q-values to 0

Step 2: Choose action (ε-Greedy policy)



Step 3: Take action and get R_{t+1}, S_{t+1}

Step 4: Update Q-table, using TD target

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha * (R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t))$$

Repeat until terminal condition

ε-Decay: Gradually lowers ε value to maximise exploitation, as training progresses

Summary

Markov Decision Process (MDP)

Base of RL: States (S), Actions (A), Rewards (R), Transition Probabilities (P), Discount γ

Value-based solving approach

V(s) – state value, **Q(s, a)** – action value, **Bellman Equations** – recursive calculation of values

Learning approaches

Monte Carlo: Learns from complete episode returns (accurate but slow)

Temporal Difference (TD): Learns after each step (faster but uses estimates)

Q-learning

Algorithm to train the **Q-function** via **Q-table**

4 Steps: Initialize → Choose action (ε-greedy) → → Execute → Observe → Update Q-value

Hands-on: Frozen lake agent



State space: 16 positions on an ice field

Action space: 4 discrete movements

Method: Q-learning Q(s, a)

GitHub link

Hands-on: Autonomous taxi



State space: 500 discrete states

Action space: 6 discrete actions: movement, taking and dropping off passenger

Method: Q-learning Q(s, a)

GitHub link